

UNIT 6 · WEEKS 8–9

Signals and shared memory

📅 Weeks 8–9 · dates to be confirmed </> Lab TD5 (solved + AI challenge) ~ 3 animations

MANUAL M6 · THEORY OF THE UNIT

Signals and shared memory

Core reading for unit 6 (≈ 1.5 h). The outputs are real: TD5 programs run on the course machine.

CONTENTS

- | | | | |
|----|--|----|---|
| §1 | Signals: sending, pending and delivery | §3 | <code>mmap</code> : mapping files into memory |
| §2 | Handlers and race-free synchronization | §4 | Shared memory and its price |

§1 Signals: sending, pending and delivery

A **signal** is an asynchronous notification to a process. It is sent by the kernel (SIGSEGV for an invalid access, SIGPIPE for writing to a pipe without a reader, SIGCHLD when a child ends), by the terminal (Ctrl-C $\rightarrow$ SIGINT, Ctrl-\ $\rightarrow$ SIGQUIT, Ctrl-Z $\rightarrow$ SIGTSTP) or by another process with `kill(pid, sig)`. For every signal, the process has an **action**: the default one (terminate, terminate with core, stop, continue or ignore), ignoring it (`SIG_IGN`) or running a **handler** function. `SIGKILL` and `SIGSTOP` are the only ones that cannot be caught, ignored or blocked.

`kill` only marks the signal as **pending**. The kernel **delivers** it when the process returns to user mode —when a system call ends or when it gets the CPU after a context switch—, unless it is **blocked** by the process's mask: then it waits. For standard signals, "pending" is a bit: if three identical ones arrive while it is blocked, only one is delivered on unblocking. Real-time signals (`SIGRTMIN` ...) do queue and can carry a value.

Practice this $\rightarrow$ A6.1 signal delivery

§2 Handlers and race-free synchronization

Handlers are installed with `sigaction` (`signal()` has different semantics across systems). While the handler runs, the same signal stays blocked (plus those in `sa_mask`); with `SA_RESTART`, an interrupted system call resumes by itself, otherwise it fails with `EINTR` (unit 2). A handler interrupts the program at any point, so it may only use *async-signal-safe* functions (`write` yes; `printf` and `malloc` no) and communicate with the program through `volatile sig_atomic_t` variables.

Waiting for a signal with `pause()` has a **race**: if the signal arrives between the last check and the call to `pause`, the handler runs early and `pause` waits forever. The fix is to **block** the signal with `sigprocmask`, check the condition and wait with `sigsuspend(mask)`, which replaces the mask and sleeps **in a single atomic operation**: the signal that came "early" stayed pending and is delivered inside `sigsuspend`.

C tds/td5/mysignal.c

```
/*
 * TD5 - exercise 1: catch SIGUSR1 with sigaction(2).
 * ./mysignal [n]      handles SIGUSR1 n times (default 3), then restores the default
action
 * From another terminal: kill -USR1 <pid>, kill -STOP <pid>, kill -CONT <pid>, kill
-KILL <pid>
 * After the n-th SIGUSR1 the default action is back: the next SIGUSR1 terminates the
process.
 */
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static volatile sig_atomic_t received = 0;

static void on_usr1(int sig)
{
    /* Only async-signal-safe functions here: no printf (it may be interrupted mid-
buffer). */
    static const char msg[] = "handler: received signal ";
    char digits[4] = { (char)('0' + sig / 10), (char)('0' + sig % 10), '\n', 0 };
    if (write_all(STDERR_FILENO, msg, sizeof msg - 1) == -1 || write_all(STDERR_FILENO,
digits, 3) == -1)
        return;
    received++;
}

int main(int argc, char *argv[])
{
    int max = argc > 1 ? atoi(argv[1]) : 3;
    struct sigaction sa = { 0 }, old;
    sigset_t block, wait_mask;

    printf("pid %d: waiting for SIGUSR1 (%d times)\n", getpid(), max);
    fflush(stdout);
```

```

sa.sa_handler = on_usr1;
sigemptyset(&sa.sa_mask);           /* nothing extra blocked while the handler runs
*/
sa.sa_flags = 0;
exit_if(sigaction(SIGUSR1, &sa, &old) == -1, "sigaction");

/* Block SIGUSR1 outside sigsuspend: a signal arriving between the test of
`received`
and the wait stays pending instead of being lost (the pause() race). */
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1, "sigprocmask");
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unblock SIGUSR1 and sleep */

exit_if(sigaction(SIGUSR1, &old, NULL) == -1, "sigaction restore");
printf("default action restored: the next SIGUSR1 terminates me\n");
fflush(stdout);
exit_if(sigprocmask(SIG_UNBLOCK, &block, NULL) == -1, "sigprocmask");
for (;;)
    pause();
}

```

Real output

```

$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep 0.2; kill -STOP $P; kill -USR1
$P; sleep 0.2; kill -CONT $P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status: $?
(128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1      ./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)

```

Other tools: `alarm(s)` schedules a `SIGALRM`; glibc's `sleep` uses `clock_nanosleep` (not `alarm`); `signalfd` turns signals into data read from a descriptor, and `pidfd_send_signal` avoids signalling a PID that has already been reused.

Practice this → TD5 · 1 and 2

§3 mmap : mapping files into memory

`mmap(addr, len, prot, flags, fd, offset)` reserves a range of addresses and ties it to a file (or to nothing, with `MAP_ANONYMOUS`). It copies nothing: each page is brought in on first access with a page fault (unit 4). With `MAP_SHARED` writes go to the file's page cache, other processes mapping it see them and they reach the disk (`msync` or on unmapping); with `MAP_PRIVATE` the first write copies the page and the file does not change. `len` is rounded up to pages; `offset` must be a multiple of the page size.

Accessing past the end of the file within the last page returns zeros that are not saved; touching a whole page with no file behind it raises **SIGBUS** (the address is valid, but there is no data to bring in). To grow a mapped file, call `ftruncate` first. `PROT_*` protections apply per page; on x86, writing implies being able to read:

Real output

```
$ ./mmap_limits
file size 128 bytes, page size 4096 bytes, mapping of 8192 bytes
wrote bytes 0..4095; byte 4096 raised SIGBUS
file size after the writes: 128 bytes
```

Real output

```
$ ./map_write
PROT = PROT_WRITE
pointer: 0x7c7095214000
read:    *p = 0
write:   *p = 42
```

Real output

```
$ ./map_exec
PROT = PROT_EXEC
pointer: 0x75b3a49d8000
bash: line 1: 357495 Segmentation fault      (core dumped) ./map_exec
[exit status 139]
```

Practice this → A6.3 the limits of mmap · TD5 · 3 and 4

§4 Shared memory and its price

Sharing memory means making virtual pages of two processes point to the same physical frame. A **rendezvous point** and an interface are needed: `mmap` of a common file with `MAP_SHARED`; POSIX `shm_open` + `mmap` (a name in `/dev/shm`, recommended today); or System V:

`ftok(file, id)` generates a key, `shmget` creates or finds the segment, `shmat` attaches it, `shmdt` detaches it and `shmctl(IPC_RMID)` destroys it. The segment outlives the processes until someone destroys it:

Real output

```
$ ./shm_counter; ./shm_counter; ./shm_counter; ./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

Two rules. First: each process sees the area at a **different address**, so pointers are not stored in it (offsets are). Second: shared memory is the fastest communication because it does not go through the kernel, but for that very reason **nothing synchronizes it**. If several processes do "read, add 1, write" on the same counter, updates are lost; with an atomic processor operation (`__atomic_add_fetch`, `lock xadd` on x86) they are not:

Real output

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Real output

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566 357567 357572 357568 357571 357569
357565 357574 357573 357570 357576 357578 357575 357577 357579
```

Practice this → A6.2 shared memory · TD5 · 5